

Hierarchical Constraint-Aware Design-Space Exploration for Heterogeneous Distributed Cyber-Physical Systems

Nikita Tarasov^a, Vasyl Tomyuk^a

^a*Lviv Polytechnic National University, Lviv, Ukraine*

Abstract

Architecting a heterogeneous distributed cyber-physical system requires simultaneous decisions on the number and class of edge and gateway nodes, on where application stages are executed, on replication, and on communication links, under conflicting latency, energy, cost and network-volume objectives. The resulting design space grows combinatorially, and for distributed cyber-physical systems a single design point is expensive to evaluate, so exhaustive exploration is impractical while population-based search returns approximate fronts without guarantees. This paper presents HCA-DSE, a hierarchical constraint-aware design-space exploration method that discards structurally ill-formed, provably infeasible and provably dominated regions of the architecture space *before* any full design-point evaluation. Its central technical ingredient is a family of admissible objective bounds for *contention-dependent* objectives: the end-to-end latency of a distributed cyber-physical system depends on tier utilisation, which is not determined by any single design decision, so a sum of assigned decision costs is not by itself an admissible bound. We prove that the resulting pruning is safe and that the exact Pareto front is preserved, and we verify both the bound admissibility and the front equality exhaustively on reference spaces. On four design-space scales, three workload classes and spaces of up to 8.3×10^7 candidate architectures, HCA-DSE performs 16–61 full evaluations instead of 10^3 to 10^7 , preserves the exhaustive Pareto front exactly (recall and precision of 1.0), and, on the largest spaces, has a visited-state-to-candidate ratio below 2×10^{-4} while never materialising the design space. Budget-matched NSGA-II requires 1000 evaluations to reach a median Pareto recall of 0.50–0.91. An evaluation-cost analysis places the measured break-even evaluator cost at 1–3 μ s on the evaluated hardware and design spaces; using the measured 19.4 ms cost of a discrete-event evaluator, the cost model projects speed-ups of up to 706 $\times$. Finally, the analytical model is validated against a containerised edge–gateway–cloud testbed with network shaping: over 36 architectures and 108 measured runs, the model orders architectures consistently with the measurements in 98% of the pooled pairs it separates by more than 25%.

Keywords: design space exploration, cyber-physical systems, edge computing, multi-objective optimisation, Pareto optimality, architecture exploration, design automation

1. Introduction

Distributed cyber-physical systems (dCPS) are increasingly deployed on a heterogeneous compute continuum that spans sensing devices, edge nodes, gateways and cloud infrastructure. Designing such a system is not a single decision but a joint one: how many edge nodes and gateways

Email address: dev.nikita@outlook.com (Nikita Tarasov)

to deploy, which hardware class each of them should be, where each application stage is executed, how far the ingest stage is replicated, and which class of communication link connects the tiers. These decisions interact. Adding edge nodes reduces queueing delay but raises deployment cost and static energy; moving analytics to the cloud removes load from the edge tier but adds a wide-area transfer to every request; replication duplicates ingest execution and multiplies both compute load and synchronisation traffic.

Because the decisions are discrete and interacting, the number of candidate architectures grows as the product of the decision domains and quickly reaches 10^5 to 10^7 even for a modest set of design variables. At the same time, evaluating a single candidate is expensive: a recent industrial dCPS exploration methodology reports that “evaluating each design point takes under a minute” with a discrete-event simulator, and its authors explicitly leave the integration of a search strategy out of scope [1]. A survey of the field concludes that “efficient and scalable DSE technology for dCPS is more or less non-existing and constitutes a largely uncharted research area” and identifies the cost of a design-point evaluation as a defining difficulty of the distributed setting [2]. The combination of a large space and an expensive evaluator is precisely the regime in which the number of *full evaluations*, rather than the raw size of the space, determines whether design-space exploration (DSE) is practical at all.

Two families of approaches dominate. Exhaustive or model-based enumeration evaluates every candidate and therefore yields the exact Pareto set, but its cost scales with the size of the space. Population-based metaheuristics such as NSGA-II [3] operate under a fixed evaluation budget and return an approximation with no guarantee on which trade-offs have been missed; as we show in Section 6, a budget-matched NSGA-II run needs 10^3 evaluations to recover half of the exact front on a space of 5×10^4 candidates. A third, smaller family performs *exact* multi-objective exploration by pruning partial designs, either symbolically with constraint solvers [4, 5] or through bound-based Pareto pruning in the broader optimisation literature [6, 7]. Assignment-monotone partial costs are one way to obtain admissible bounds, but exact bound-based search does not require all objectives to be assignment monotone. Consistent approximations have also been used to avoid expensive DSE evaluations while retaining exactness [8].

Here we study a concrete contention-dependent architecture model. Latency contains a waiting component governed jointly by node counts, resource classes, replication and placement. We derive objective-specific relaxations for this model and quantify their effect on exploration. The contribution is the model, bound construction and empirical analysis, rather than a new general principle of safe Pareto pruning.

Contributions.

1. A formal design-space model for heterogeneous edge–gateway–cloud CPS architectures (Section 3) with analytical models of latency including contention, energy, normalised deployment cost, network volume, and six classes of feasibility constraints.
2. **Admissible bounds for contention-dependent objectives** (Section 4). We decompose each objective into non-negative terms, bound every term independently over the still-reachable domains, and include the waiting term exactly once its determinants are fixed. In the evaluated variable order this occurs at full assignment; earlier prefixes use a zero waiting bound. Lemmas 1 and 2 establish admissibility.
3. **HCA-DSE**, a hierarchical exploration method that combines structural well-formedness, feasibility pruning and bound-based dominance pruning with a final exact evaluation of the

survivors, together with proofs (Propositions 1–3, Theorem 1) that the exact Pareto front is preserved.

4. **An evaluation-cost analysis** that answers when such pruning is worth its own overhead, including a measured break-even evaluator cost and a crossover curve (Section 6).
5. **An empirical validation** on a containerised edge–gateway–cloud testbed with per-node link shaping: 36 architectures, 108 measured runs, reported both as prediction error and as the agreement between the architecture ordering produced by the model and the ordering observed in the measurements.
6. A reproducibility package in which every figure and every numeric table of this paper is generated from the raw result files by a single script, and in which the correctness claims are backed by exhaustive machine-checked tests.

The remainder of the paper is organised as follows. Section 2 positions the work. Section 3 defines the design space and the evaluation model. Section 4 presents HCA-DSE and its correctness. Section 5 describes the experimental methodology, Section 6 the results. Section 7 discusses when the method pays off and when it does not, Section 8 the threats to validity, and Section 9 concludes.

2. Related work

2.1. Exact multi-objective design-space exploration

The closest methodological relatives of this work perform exact multi-objective exploration by reasoning about partial designs. Neubauer et al. [4] encode system-level synthesis in Answer Set Programming modulo Theories and attach theory propagators that check both constraints and Pareto dominance on *incomplete* assignments, returning conflict clauses that remove infeasible and dominated regions early. Their formulation targets heterogeneous multiprocessors with routing, optimising latency, area and energy, and their pruning is sound for problems whose objectives are assignment monotonous. In the broader optimisation literature, multi-objective A^* with admissible or consistent heuristics [9, 7] provides its theoretical foundation for bound-based pruning: a partial solution whose optimistic vector is dominated by a complete solution can be discarded without losing Pareto-optimal completions.

HCA-DSE belongs to this family, and we do not claim safe pruning as such as a contribution. The difference is the class of objectives. In a distributed CPS the dominant latency term is contention-dependent: it is a function of tier utilisation, which no single assignment determines. We therefore construct bounds that are admissible without assignment monotonicity, by combining a term-wise relaxation with an explicit decision ordering that makes the contention term exactly computable as soon as its determinants are fixed (Section 4). Avoiding expensive evaluations is also an explicit aim of consistent-approximation DSE [8], which combines safe approximations with exact assessment of promising designs. We therefore do not claim evaluator avoidance as a new principle. Our study specialises bounds to this CPS model and measures the evaluation-cost trade-off. The term “Pareto pruning” also refers to selecting a representative subset of an already computed front [6]; that operation differs from our exact search-space pruning.

2.2. Design-space exploration for distributed cyber-physical systems

Herget et al. [2] survey DSE for dCPS and identify the two premises of this paper: design spaces that are far larger than in classical MPSoC exploration, and design-point evaluations that are considerably more expensive. CompDSE [1] follows the Y-chart philosophy [10] and derives

workload, platform and mapping models from execution traces of an industrial lithography machine, evaluating design points with OMNeT++ [11]. It is an evaluator, not a search strategy: the authors report a manual exploration of three variations and state that integrating search algorithms is out of scope. The two approaches are therefore complementary — CompDSE answers how a design point should be evaluated, HCA-DSE answers which design points need not be evaluated at all — and the per-design-point cost reported there falls in the regime where our crossover analysis predicts speed-ups of two orders of magnitude.

ContrArc [12] explores CPS architectures with a mixed-integer program combined with contract refinement checking, pruning through subgraph isomorphism. It minimises a single objective (cost) subject to requirements rather than constructing a Pareto front, and its pruning is constraint-based rather than bound-based.

2.3. Exact allocation by integer programming

A second exact family formulates allocation as an integer program and solves it to optimality. Kouloumpris et al. [13] allocate the tasks of an application over an edge/hub/cloud triple with binary integer linear programming, minimising latency or energy under computation, communication, memory and energy constraints, and validate on a UAV inspection use case with synthetic benchmarks of up to 1000 tasks. Their follow-up [14] extends the formulation to a bi-objective reliability/latency problem, again solved exactly and offline.

These works are exact in a different sense than the present paper, and the distinction matters for the choice of method. They solve one *allocation* instance over a fixed infrastructure with a model that is linear in the decision variables, whereas we explore an *architecture* space in which the number and class of nodes, the link class and the replication factor are themselves decisions, and in which the latency model contains a non-linear contention term. A compact mixed-integer linear formulation would require an encoding of the queueing term. Piecewise-linear approximations can change the objective, but finite-domain table or disjunctive encodings can in principle retain the exact values at the cost of model size. Nonlinearity therefore does not rule out an exact solver baseline; its relative efficiency remains untested here.

2.4. Design-time placement in the edge–cloud continuum

SPACE4AI-D [15] is the closest work in terms of the problem setting: design-time selection of resources and component placement across edge, cloud and function-as-a-service layers. The problem is formulated as a mixed-integer non-linear program with a single objective (deployment cost) under response-time constraints, and solved with randomised greedy, local search, tabu search, simulated annealing and genetic algorithms; no exactness guarantee is provided. Its validation on a real deployment reports deviations of 12% in cost and 20% in execution time, which is a useful reference point for the accuracy reported in Section 6.

2.5. Metaheuristic exploration of edge–cloud deployments

A large body of work applies evolutionary and swarm metaheuristics to task allocation, offloading and service placement in edge–cloud systems, typically optimising combinations of latency, energy and cost. These methods are approximate by construction and their quality is a function of the evaluation budget. We use NSGA-II [3] as implemented in pymoo [16] as the budget-matched representative of this family, and report hypervolume [17], IGD⁺ [18] and Pareto recall against the exhaustive ground truth; the choice of indicators follows the guidance of Audet et al. [19]. Our

Table 1: Positioning with respect to the closest prior work. “Partial-design pruning” denotes discarding incomplete designs before they are completed and evaluated; “contention-aware” denotes objectives that depend on resource utilisation rather than being additive over decisions.

Work	Year	System class	Design variables	Obj.	Exact front	Partial-design	Cont.-aware	Exp. eval.	Real meas.
Blickle et al. [20]	1998	embedded synthesis	allocation, binding	2	no	no	no	no	no
Haubelt & ich [21]	Te-2003	SoC	component selection	n	partly	front comp.	no	no	no
Lukasiewicz et al. [22]	2008	embedded synthesis	allocation, binding, routing	n	no	SAT-based	no	no	no
Neubauer et al. [4]	2018	heterog. MPSoC	allocation, binding, routing	3	yes	symbolic	no	no	no
Herget et al. [2]	2022	distributed CPS	survey	—	—	—	—	—	—
Kouloumpiris et al. [13]	2024	edge/hub/cloud	task allocation	1	yes (BILP)	solver	no	no	UAV case
Xiao et al. [12]	2024	CPS architectures	component selection	1	—	isomorphism	no	no	no
Sedghani et al. [15]	2024	computing continuum	resources, placement	1	no	no	partly	no	yes
CompDSE [1]	2025	industrial dCPS	cores, hosts, frequency	perf.	—	none	yes	—	traces
Kouloumpiris et al. [14]	2026	edge/hub/cloud	task allocation	2	yes (BILP)	solver	no	no	case study
NSGA-II family [3]	—	edge-cloud	varies	2–4	no	no	partly	no	rarely
HCA-DSE (this work)	2026	heterog. edge-cloud CPS	counts, classes, links, placement, replication	4	yes	bounds	yes	yes	yes

conclusion is not that such methods are inferior in general, but that when admissible bounds are available exactness is achievable at a small fraction of their evaluation budget.

Table 1 summarises the comparison.

3. System and design-space model

3.1. Architecture template

We consider the four-tier template of Fig. 1. A population of devices generates requests; each request traverses three application stages. The *pre-processing* stage τ_1 runs either on the edge tier or on the gateway tier, the *aggregation* stage τ_2 always runs on the gateway tier, and the *analytics* stage τ_3 runs either on the edge tier or in the cloud. Which of these variants is instantiated is a design decision (the placement policy).

A compute node class is a tuple $(C, M, P^{\text{idle}}, P^{\text{peak}}, c)$ of compute capacity [MIPS], memory [MB], idle and peak power [W] and normalised cost per node. A link class is a tuple $(B, \tau^{\text{prop}}, e, c)$ of bandwidth [Mb/s], propagation delay [s], transfer energy [J/MB] and cost per attached node. The gateway-to-cloud connection is fixed infrastructure ($B = 200$ Mb/s, $\tau^{\text{prop}} = 20$ ms) and is not a design variable. Table 2 lists the concrete classes used in the experiments.

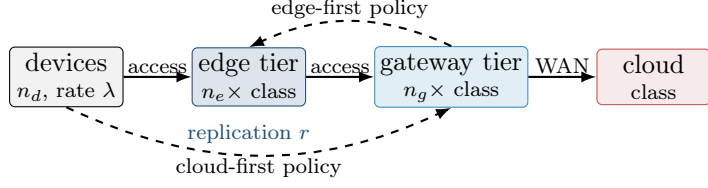


Figure 1: Architecture template. Solid arrows show the default request path; dashed arrows show the alternative stage placements selected by the placement policy.

Table 2: Resource classes. Compute classes: capacity C [MIPS], memory M [MB], idle and peak power [W], cost [units]. Link classes: bandwidth B , propagation delay τ , transfer energy e , cost.

class	C / B	M / τ	P^{idle} / e	P^{peak}	cost
<i>compute node classes</i>					
edge-low	2000	2048	4	12	1
edge-med	6000	8192	8	30	2.4
edge-high	15000	16384	15	65	5.2
gw-low	4000	4096	6	20	1.8
gw-high	12000	16384	12	48	4
cloud-med	40000	131072	0	180	6
cloud-high	100000	262144	0	380	12
<i>link classes</i> (name, B [Mb/s], τ [ms], e [J/MB], cost)					
constrained	10	4	0.3	0.2	–
standard	100	2	0.12	0.6	–
fast	1000	1	0.05	1.4	–

Table 3: Workload classes. Stage values are given as $\tau_1/\tau_2/\tau_3$.

workload	n_d	λ	Λ	w_i [MI]	s_i [kB]	D [ms]	C^{max}
telemetry	200	2	400	8/4/20	20/10/10	120	60
control	60	10	600	5/3/9	5/4/3	30	60
sensing	40	1.5	60	120/30/260	1200/350/300	900	90

3.2. Workload

A workload is $(n_d, \lambda, \{\tau_i\}_{i=1}^3, D, s^{\text{sync}})$ where n_d devices each emit requests at rate λ , giving a system arrival rate $\Lambda = n_d \lambda$; stage $\tau_i = (w_i, m_i, s_i)$ has computational work w_i [MI], resident memory m_i [MB] and inbound payload s_i [MB]; D is the end-to-end deadline and s^{sync} the per-request replica synchronisation payload. Three workload classes are used throughout (Table 3); they are qualitatively different rather than numerically rescaled versions of one another: telemetry is high-rate and small-payload, real-time control is deadline-dominated, and data-intensive sensing is payload- and compute-dominated.

3.3. Decision vector and design space

An architecture is described by the vector

$$x = (n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi), \quad (1)$$

with n_e, n_g the number of edge nodes and gateways, $\kappa_e, \kappa_g, \kappa_c$ the edge, gateway and cloud classes, ℓ the access link class, r the replication factor of stage τ_1 and π the placement policy. The design space is the Cartesian product $\mathcal{X} = D_1 \times \dots \times D_k$ with $k = 8$, and $|\mathcal{X}| = \prod_i |D_i|$. Table 4 lists the domains of the four scales used in the experiments.

Table 4: Design variables and domain sizes of the four design-space scales.

variable	meaning	S1	S2	S3	S4
n_e	number of edge nodes	3	6	24	80
$edge_cls$	edge node class	3	3	3	3
n_g	number of gateways	2	3	5	8
gw_cls	gateway class	2	2	2	2
$link_cls$	access link class	3	3	3	3
$cloud_cls$	cloud instance class	2	2	2	2
rep	replication factor of stage 1	2	3	4	5
$policy$	placement policy	3	3	3	3
<i>design-space size</i> $ \mathcal{X} $		1 296	5 832	51 840	345 600

3.4. Evaluation model

Let $t(i)$ denote the tier executing stage i under policy π , N_t the number of nodes of tier t and C_t the per-node capacity of its class. The offered load and utilisation of a tier are

$$\Theta_t = \sum_{i:t(i)=t} \Lambda w_i \mu_i, \quad \rho_t = \frac{\Theta_t}{N_t C_t}, \quad \mu_i = \begin{cases} r & i = 1 \\ 1 & \text{otherwise,} \end{cases} \quad (2)$$

that is, replication multiplies the load of the ingest stage but not of the others.

Latency. We use a stage-wise M/D/1-inspired waiting approximation [23, 24]: each stage retains its per-node service time $w_i/C_{t(i)}$, while the waiting multiplier uses the average tier utilisation ρ_t based on aggregate capacity $N_t C_t$. This is not an exact M/D/1 queue with service capacity $N_t C_t$: such a queue would also divide the service time by N_t . Nor is it an exact multi-server model. Stages sharing a tier have different service demands, and routing, replication and successive visits can violate the Poisson-arrival assumption. The expression is therefore a tractable surrogate whose prediction error must be assessed empirically; it is not a proven lower bound on deployment latency. The simulator and testbed implement per-node servers and quantify this gap. With $\mathcal{H}(x)$ the set of communication hops implied by the policy and the replication factor,

$$L(x) = \underbrace{\sum_{i=1}^3 \frac{w_i}{C_{t(i)}}}_{\text{service}} + \underbrace{\sum_{i=1}^3 \frac{w_i}{C_{t(i)}} \cdot \frac{\rho_{t(i)}}{2(1-\rho_{t(i)})}}_{\text{waiting}} + \underbrace{\sum_{(s,\ell) \in \mathcal{H}(x)} \left(\frac{8s}{B_\ell} + \tau_\ell^{\text{prop}} \right)}_{\text{communication}}. \quad (3)$$

The middle term is what makes the objective *contention-dependent*: it is a function of ρ_t , which by (2) depends jointly on n_t , the class of the tier, the replication factor and the policy. If $\rho_t \geq 1$ for any used tier the architecture is discarded as unstable.

Energy, cost and network volume. Over an accounting horizon T ,

$$E(x) = \sum_{t \in \{e,g\}} N_t \left(P_t^{\text{idle}} + \rho_t (P_t^{\text{peak}} - P_t^{\text{idle}}) \right) T + \rho_c P_c^{\text{peak}} T + \sum_{(s,\ell) \in \mathcal{H}(x)} s \Lambda e_\ell T, \quad (4)$$

$$C(x) = n_e (c_{\kappa_e} + c_\ell) + n_g (c_{\kappa_g} + c_\ell) + c_{\kappa_c}, \quad (5)$$

$$D(x) = \Lambda \sum_{(s,\ell) \in \mathcal{H}(x)} s. \quad (6)$$

The objective vector is $F(x) = (L(x), E(x), C(x), D(x))$, all components minimised. Energy is the total consumption over the fixed accounting horizon $T = 1$ h and is reported in kWh; the cost coefficients of Table 2 are synthetic normalised values encoding the relative price tiers of the resource classes, and no monetary interpretation is claimed. Neither the energy nor the cost model is validated against measurements in this paper: they are used for comparative exploration only.

Feasibility. An architecture is feasible when all of the following hold: $\rho_t \leq \rho^{\max}$ for every used tier; the resident memory of the stages mapped to a tier fits in the memory of its node class; the aggregate access-link traffic does not exceed the total bandwidth of the attached nodes; $L(x) \leq D$; $C(x) \leq C^{\max}$; and $E(x) \leq E^{\max}$. We write $\mathcal{F}$ for the set of feasible architectures.

4. HCA-DSE

4.1. Partial architectures

The exploration traverses a prefix tree over the decision vector (1) under a fixed variable order $v_1, \dots, v_k$.

Definition 1 (Partial architecture). A partial architecture of depth d is a prefix of assignments $p = (a_1, \dots, a_d)$, $a_i \in D_i$. Its completions are $\mathcal{C}(p) = \{x \in \mathcal{X} : x_i = a_i, i \leq d\}$, with $|\mathcal{C}(p)| = \prod_{i>d} |D_i|$.

Let $\sigma : \mathcal{X} \rightarrow \{0, 1\}$ be the well-formedness predicate of the template: the number of gateways does not exceed the number of edge nodes, the replication factor does not exceed the number of edge nodes, and replication is undefined for policies that do not place the ingest stage on the edge tier. The working space is $\mathcal{X}_\sigma = \{x : \sigma(x) = 1\}$ and all statements below refer to $\mathcal{F}_\sigma = \mathcal{F} \cap \mathcal{X}_\sigma$; the exhaustive baseline against which we compare uses the same predicate. The Pareto front is

$$\mathcal{P}^* = \{F(x) : x \in \mathcal{F}_\sigma, \nexists y \in \mathcal{F}_\sigma : F(y) \preceq F(x)\}, \quad (7)$$

where $u \preceq v$ denotes $u_i \leq v_i$ for all i and $u_j < v_j$ for at least one j .

4.2. Admissible bounds

Definition 2 (Admissibility). LB_i is admissible for objective f_i if $LB_i(p) \leq f_i(x)$ for every partial architecture p and every $x \in \mathcal{C}(p)$.

The partial-cost construction discussed by Neubauer et al. assumes that objectives are *assignment monotonous*, i.e. that every assignment can only increase the objective, so that the partial cost is itself a bound [4]. The waiting term of (3) violates this assumption: it depends on ρ_t , which by (2) is jointly determined by n_t , the class of the tier, r and π , and which *decreases* when nodes are added. We therefore construct the bounds in two steps.

Lemma 1 (Term-wise relaxation). Let $f_i(x) = \sum_{t \in T_i} \varphi_t(x)$ with $\varphi_t \geq 0$, and let

$$\underline{\varphi}_t(p) = \min_{x \in \mathcal{C}(p)} \varphi_t(x).$$

Then $LB_i(p) = \sum_{t \in T_i} \underline{\varphi}_t(p)$ is admissible.

Proof. For every $x \in \mathcal{C}(p)$ and every t , $\underline{\varphi}_t(p) \leq \varphi_t(x)$ by definition of the minimum. Summing over $t \in T_i$ yields $LB_i(p) \leq f_i(x)$. $\square$

The minima are taken *independently per term*, so the combination of values that realises them need not correspond to any single completion; the bound is therefore optimistic but valid. The implementation uses domain extrema for service, communication, static energy and cost, and minima of class-specific power-to-capacity ratios for dynamic energy. Categorical policies are enumerated over their small reachable set; they have no meaningful monotone endpoint order. Independent extrema can relax a term further than its exact minimum, which remains safe. The queueing contribution is bounded by zero until its determinants are fixed. Domain aggregates are precomputed and repeated class-dependent terms cached; cache misses can still require scans of reachable classes or policies.

Lemma 2 (Exactness on determined terms). *Let $\text{supp}(\varphi_t)$ be the set of variables φ_t depends on. If $\text{supp}(\varphi_t) \subseteq \{v_1, \dots, v_d\}$ then φ_t is constant on $\mathcal{C}(p)$, and substituting its exact value for $\varphi_t(p)$ preserves admissibility while tightening the bound.*

Proof. Constancy is immediate, and a constant equals its own minimum over $\mathcal{C}(p)$, so Lemma 1 applies unchanged. $\square$

Remark 1 (Why the variable order matters). Once all determinants of a tier’s waiting contribution are assigned, that contribution is constant on $\mathcal{C}(p)$ and can enter the bound exactly by Lemma 2; before then, zero is admissible for stable completions. If a determined used tier has $\rho_t \geq 1$, its latency is infinite. In the evaluated implementation the order is $(n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi)$. Because policy is assigned last, the exact queueing contribution is activated only at depth eight, before full evaluation. The reported experiments therefore do not demonstrate early queueing-based pruning. Moving policy and replication earlier could activate this contribution on incomplete designs, but the effect on pruning efficiency requires a separate experiment.

4.3. Pruning rules

Proposition 1 (Safe feasibility pruning). *Let LB_{g_j} be admissible for constraint function g_j . If $LB_{g_j}(p) > g_j^{\max}$ then $\mathcal{C}(p) \cap \mathcal{F} = \emptyset$.*

Proof. For any $x \in \mathcal{C}(p)$, $g_j(x) \geq LB_{g_j}(p) > g_j^{\max}$, hence $x \notin \mathcal{F}$. $\square$

Proposition 2 (Safe dominance pruning). *Let $y \in \mathcal{F}_\sigma$ be a fully evaluated feasible architecture with $F(y) \preceq LB(p) = (LB_1(p), \dots, LB_n(p))$. Then $F(x) \notin \mathcal{P}^*$ for every $x \in \mathcal{C}(p)$.*

Proof. Let $x \in \mathcal{C}(p)$. By admissibility $f_i(x) \geq LB_i(p) \geq F_i(y)$ for all i , and by assumption there is a j with $F_j(y) < LB_j(p) \leq f_j(x)$. Hence y strictly dominates x and $F(x)$ is not on the Pareto front. $\square$

Proposition 3 (Closure of the well-formedness predicate). *Each invariant of σ depends only on assigned variables and, under the chosen order, all variables of an invariant are assigned before it is first checked. Since assignments are never revised, $\neg\sigma(p)$ implies $\sigma(x) = 0$ for every $x \in \mathcal{C}(p)$, i.e. $\mathcal{C}(p) \cap \mathcal{X}_\sigma = \emptyset$.*

Remark 2. Structural invariants define what counts as an architecture rather than following from the constraints of the problem: the evaluator can formally compute objectives for, say, a replicated deployment under a policy that does not place the ingest stage on the edge tier, although such a configuration is meaningless. Structural pruning is therefore part of the definition of the search space, and the exhaustive baseline applies the same predicate. This distinction was found by the test described in Section 5, which initially — and incorrectly — verified structural pruning as a feasibility argument.

Algorithm 1 HCA-DSE

Require: design space $\mathcal{X}$ with order $v_1..v_k$, workload w , evaluator F

Ensure: Pareto front $\mathcal{P}$

```

1:  $A \leftarrow \emptyset$  ▷ archive of non-dominated evaluated vectors
2:  $S \leftarrow \emptyset$  ▷ evaluated feasible architectures
3: procedure EXPLORE( $p, d$ )
4:   if  $d = k$  then
5:      $(F(p), \text{feasible}) \leftarrow \text{EVALUATE}(p, w)$  ▷ the expensive step
6:     if  $\text{feasible}$  then  $S \leftarrow S \cup \{p\}$ ; ARCHIVEINSERT( $A, F(p)$ )
7:     end if
8:     return
9:   end if
10:  for all  $a \in D_{d+1}$  do
11:     $q \leftarrow p \cup \{v_{d+1} \mapsto a\}$ 
12:    if  $\neg \sigma(q)$  then continue ▷ Prop. 3
13:    end if
14:     $LB \leftarrow \text{BOUNDS}(q)$  ▷ Lemmas 1, 2
15:    if  $\exists j : LB_{g_j}(q) > g_j^{\max}$  then continue ▷ Prop. 1
16:    end if
17:    if  $\exists f \in A : f \preceq LB$  then continue ▷ Prop. 2
18:    end if
19:    EXPLORE( $q, d + 1$ )
20:  end for
21: end procedure
22: EXPLORE( $\emptyset, 0$ )
23: return non-dominated subset of  $\{F(x) : x \in S\}$ 

```

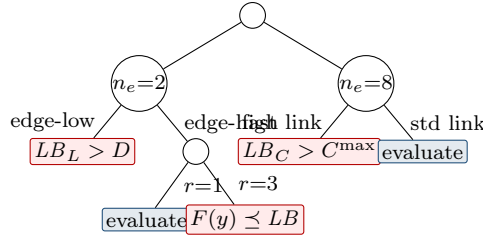


Figure 2: Pruning on the prefix tree. Sub-trees are discarded when the optimistic bound already violates a constraint (Proposition 1) or is dominated by an evaluated feasible design (Proposition 2); only surviving leaves reach the expensive evaluator.

4.4. Algorithm

Algorithm 1 traverses the prefix tree depth-first. At every node the three rules are applied in increasing order of cost: structural validity, then feasibility bounds, then dominance against an archive of non-dominated evaluated points. A leaf that survives is evaluated in full; feasible leaves update the archive. Fig. 2 illustrates the two bound-based rules.

4.5. Correctness

Theorem 1 (Pareto-front preservation). *If all bounds are admissible, pruning is performed only by Propositions 1–3, and the archive contains only feasible fully evaluated architectures, then the set of non-dominated vectors returned by Algorithm 1 equals the true Pareto front: $\mathcal{P}^{\text{HCA}} = \mathcal{P}^*$.*

Proof. ($\subseteq$) Every returned vector is feasible. If a returned vector were dominated by any feasible vector, finiteness of $\mathcal{X}$ implies that it is dominated by a Pareto-optimal vector. The following argument shows that every Pareto-optimal vector is evaluated; it would therefore remove the dominated vector during final filtering, a contradiction.

($\supseteq$) Let $u \in \mathcal{P}^*$ and let $x \in \mathcal{F}_\sigma$ with $F(x) = u$. The traversal is a prefix tree, so x is reached through the unique chain $p_0 \subset p_1 \subset \dots \subset p_k = x$. Suppose some p_d were pruned. By Proposition 3 this implies $\sigma(x) = 0$ and by Proposition 1 $x \notin \mathcal{F}$, both contradicting $x \in \mathcal{F}_\sigma$; by Proposition 2 it implies the existence of a feasible y strictly dominating x , contradicting $u \in \mathcal{P}^*$. Hence no p_d is pruned, x is evaluated, and u survives the final filtering. $\square$

Remark 3 (Duplicates). The theorem is stated in objective space. The strict dominance rule does not prune a design merely because its objective vector equals an archived vector. A separate duplicate-removal step may retain one representative per vector without changing the front. All experiments therefore compare sets of objective vectors.

Remark 4 (Order independence). Proposition 2 holds for any archive of evaluated feasible architectures, so the traversal order affects pruning efficiency but never correctness.

4.6. Complexity

Let C_E be the cost of one full evaluation, C_P the cost of computing the bounds at one tree node, N the number of visited partial states, N_E the number of full evaluations and $|A|$ the archive size. Then

$$T_{\text{exh}} = O(|\mathcal{X}| C_E), \quad T_{\text{HCA}} = O(N(C_P + n|A|) + N_E C_E), \quad (8)$$

and pruning is profitable when

$$C_E > \frac{N(C_P + n|A|)}{|\mathcal{X}| - N_E}. \quad (9)$$

Section 6 measures the right-hand side and shows that it lies in the microsecond range, and that $N \ll |\mathcal{X}|$, so the Cartesian product is never materialised.

5. Experimental methodology

5.1. Research questions

RQ1 By how much does HCA-DSE reduce the number of *full design-point evaluations*, and how is that reduction distributed over the pruning layers?

RQ2 Is the exact Pareto front preserved with respect to exhaustive enumeration?

RQ3 At which evaluator cost does the pruning overhead pay for itself?

RQ4 How does HCA-DSE compare with budget-matched stochastic exploration?

RQ5 How do the reduction and the runtime scale with the size of the design space?

RQ6 Do the architecture rankings produced by the analytical model transfer to a real containerised edge–cloud deployment?

5.2. Design of the experiments

Four design-space scales (S1–S4, Table 4) are combined with the three workload classes of Table 3. Exhaustive ground truth is computed for S1–S3; S4 and the scalability study are explored by HCA-DSE only. Every configuration is deterministic except the stochastic baselines, which are run with 20 independent seeds.

Baselines. Exhaustive enumeration provides the ground-truth Pareto front. Random search and NSGA-II [3] (pymoo [16], population 40, SBX crossover, polynomial mutation, integer repair) are run under matched evaluation budgets of 100, 250, 500 and 1000 full evaluations, 20 seeds each. All methods use exactly the same evaluator and the same feasibility rules, so differences are attributable to the exploration strategy alone.

Metrics. Pareto recall and precision are computed on sets of objective vectors against the exhaustive front. Hypervolume [17] is normalised by the hypervolume of the exhaustive front (HV ratio); IGD^+ [18] uses the exhaustive front as reference. We additionally report the number of full evaluations N_E , the number of visited partial states N , and wall-clock time. Stochastic baselines are compared with the Mann–Whitney U test [25] and Cliff’s δ [26]. The Mann–Whitney alternative is that NSGA-II has larger HV values than random search; positive δ favours NSGA-II. Reported p -values are unadjusted and comparisons are exploratory.

Ablation. Four variants isolate the contribution of each layer: structural only; structural + feasibility bounds; structural + dominance bounds; and the full method.

5.3. Two-level evaluation and the cost model

The analytical evaluator of Section 3 (level A) is used for pruning, ground truth and all exactness claims. A discrete-event simulator (level B) provides a higher-fidelity, and therefore more expensive, evaluator: multi-server queues with a finite number of servers per tier, explicit link contention, deterministic service times and Poisson arrivals. Its measured cost is 19.4 ms per design point against $\approx 8 \mu\text{s}$ for the analytical evaluator, which allows the crossover analysis of RQ3 to be anchored at two measured points rather than assumed.

5.4. Testbed

The physical validation runs the architecture template as containers (Fig. 3). Each edge node, gateway and cloud instance is a separate container limited to one CPU, so that a node behaves as the single server assumed by the model. Services are implemented in Go and share one binary: a request carries a plan listing the remaining stages, and each node consumes CPU work equal to w_i/C_t before forwarding the remainder together with the real payload to the next hop. Links are shaped per node with `tc netem` [27] using the delay and rate of the link class of the design under test; the gateway-to-cloud hop uses the WAN profile. Replication is emulated faithfully: r replicas execute stage τ_1 in parallel and carry the synchronisation payload, while the remainder of the chain is executed once. A load generator produces Poisson arrivals at rate Λ , 900 requests per run of which the first 150 are discarded as warm-up, and reports the latency percentiles.

Thirty-six architectures were selected across the three workloads — Pareto-optimal, dominated and near-boundary designs — restricted to those that fit on a single machine (≤ 4 edge nodes, ≤ 2 gateways, one cloud instance). Each was executed three times, giving 108 measured runs.

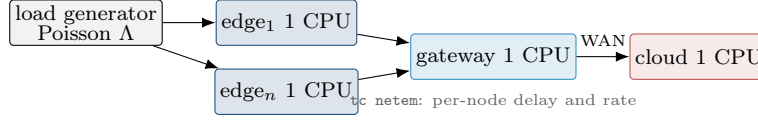


Figure 3: Containerised testbed. Every tier node is a separate one-CPU container; access links carry the delay and bandwidth of the link class of the design, the cloud hop carries the WAN profile.

Because the analytical model is deliberately coarse, we report both the prediction error (MAPE, median absolute error) and, more importantly for exploration, the *ordering* agreement: Spearman’s ρ , Kendall’s τ , and the share of architecture pairs whose measured order agrees with the model order among the pairs that the model separates by more than 10% and 25%.

5.5. Reproducibility

The implementation is in Python 3 with NumPy, SciPy and pymoo; the testbed services are in Go and run under Docker Compose. All figures and all numeric tables of this paper are generated from the raw result files by a single script, so no number is transcribed by hand. Correctness is machine-checked at three levels: (i) bound admissibility is verified *exhaustively* on S1, i.e. for every prefix and every completion of that prefix across the three workloads; (ii) for every pruned prefix, all of its completions are enumerated and the claim of the corresponding proposition is verified directly, on S1 and S2; and (iii) the equality $\mathcal{P}^{\text{HCA}} = \mathcal{P}^{\star}$ is re-checked on S1–S3 for every pruning variant as a regression test.

6. Results

6.1. RQ1: most architectures are discarded before evaluation

Table 5 and Fig. 4 report the exploration funnel. On S3 (51 840 candidates) HCA-DSE performs 41 to 54 full evaluations depending on the workload, a reduction of 99.90% with respect to the raw space; on S4 (345 600 candidates) it performs 52 to 60, a reduction of 99.98%. The composition of the funnel differs by workload: for data-intensive sensing the feasibility bounds remove the largest share, whereas for telemetry dominance pruning does most of the work, which reflects the different binding constraints of the two regimes.

Table 5: Exploration funnel. The reduction column refers to *full design-point evaluations* with respect to the raw design space, not to runtime; recall is against exhaustive enumeration where the latter is tractable.

scale	workload	$ \mathcal{K} $	structural pruned	feasibility pruned	dominance pruned	full evals	feasible	$ \mathcal{P} $	full-eval reduction [%]	exhaustive evals	recall
S1	telemetry	1 296	138	233	893	32	25	17	97.53	1 080	1.0
S1	control	1 296	138	269	863	26	23	18	97.99	1 080	1.0
S1	sensing	1 296	77	613	590	16	11	11	98.77	1 080	1.0
S2	telemetry	5 832	1 422	1 127	3 235	48	40	27	99.18	3 888	1.0
S2	control	5 832	1 425	1 236	3 133	38	35	25	99.35	3 888	1.0
S2	sensing	5 832	1 075	2 323	2 408	26	19	16	99.55	3 888	1.0
S3	telemetry	51 840	8 374	27 573	15 839	54	46	31	99.90	34 560	1.0
S3	control	51 840	8 385	28 512	14 902	41	38	27	99.92	34 560	1.0
S3	sensing	51 840	7 176	27 328	17 293	43	36	27	99.92	34 560	1.0
S4	telemetry	345 600	24 331	292 656	28 553	60	52	32	99.98	–	–
S4	control	345 600	24 351	294 344	26 853	52	49	29	99.98	–	–
S4	sensing	345 600	23 497	286 394	35 657	52	45	28	99.98	–	–

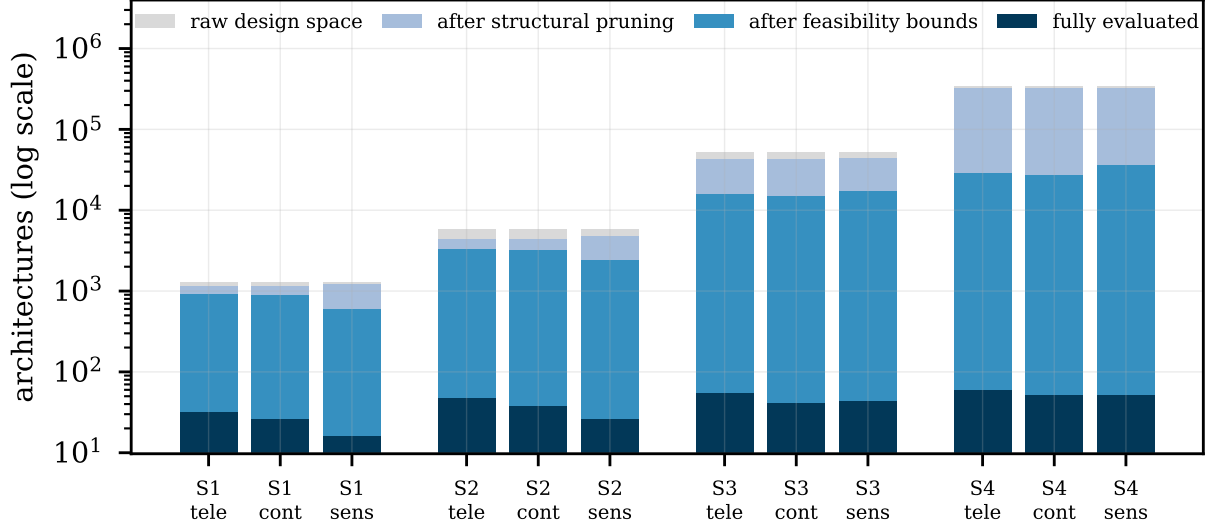


Figure 4: Design-space funnel across scales and workloads (logarithmic scale).

6.2. RQ2: the exact front is preserved

For every scale for which exhaustive enumeration is tractable (S1–S3) and for every workload, the set of objective vectors returned by HCA-DSE is identical to the exhaustive Pareto front: recall and precision are both 1.0. The same holds for each ablated variant, confirming that no individual pruning layer is unsafe. Bound admissibility was verified exhaustively on S1 — all prefixes against all completions — without a single violation, and for every pruned prefix on S1 and S2 all completions were enumerated and found consistent with the claim of the corresponding proposition. Fig. 5 shows the resulting front for S3/telemetry in two projections.

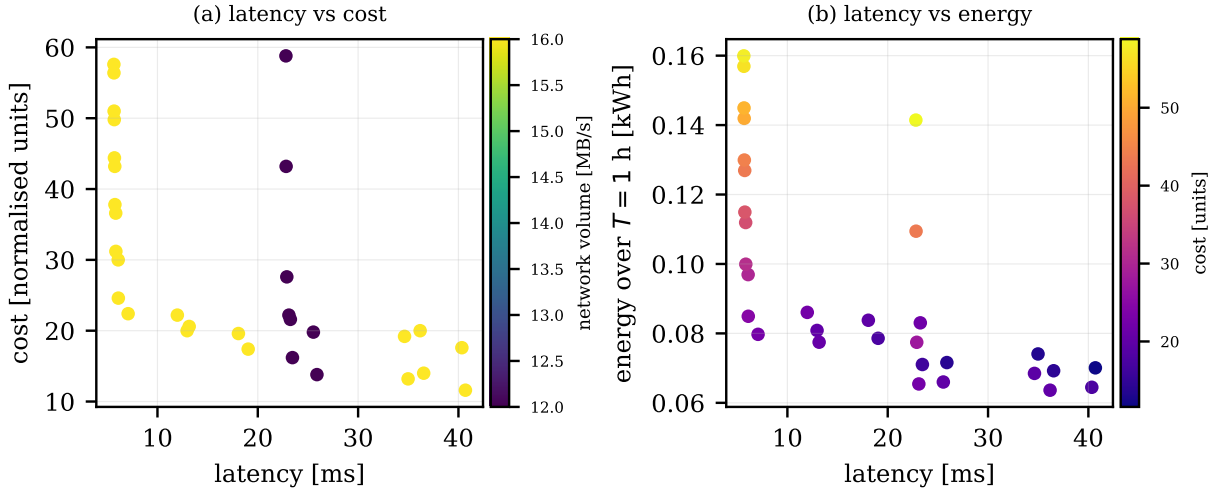


Figure 5: Exact Pareto front recovered on S3/telemetry. (a) latency versus cost, colour encodes network volume; (b) latency versus energy over the accounting horizon $T = 1$ h, colour encodes cost.

6.3. RQ3: when pruning pays off

Using the measured overheads and evaluation counts in (9), the break-even evaluator cost is $1\text{--}3\mu\text{s}$ on S2 and below $1\mu\text{s}$ on S3. This figure is a property of the evaluated implementation, hardware, design-space geometry and search order, not a universal constant; what it establishes is that on the spaces studied here the pruning overhead is repaid by evaluators whose cost is of the order of microseconds and above, which places the discrete-event evaluator used in this paper well inside the favourable regime. Fig. 6 shows the speed-up over exhaustive enumeration as a function of C_E . We stress that the curve is a *projection*: the overheads and the evaluation counts are measured, and the cost of one discrete-event evaluation (19.4ms , averaged over 40 design points) is measured, but the two exploration strategies were not re-executed end-to-end with the discrete-event evaluator in the inner loop. Using the saved overheads at $C_E = 19.4\text{ms}$, projected speed-ups range from $77.5\times$ to $143.1\times$ on S2 and from $544.8\times$ to $706.2\times$ on S3 across workloads. The corresponding asymptotic evaluation-count ratios range from $81.0\times$ to $149.5\times$ and from $640.0\times$ to $842.9\times$, respectively. With the cheap analytical evaluator the measured speed-up is only $1.3\text{--}3.3\times$, because then the bound computation dominates the runtime; this is exactly the regime in which pruning is *not* the right tool, and we report it rather than omit it.

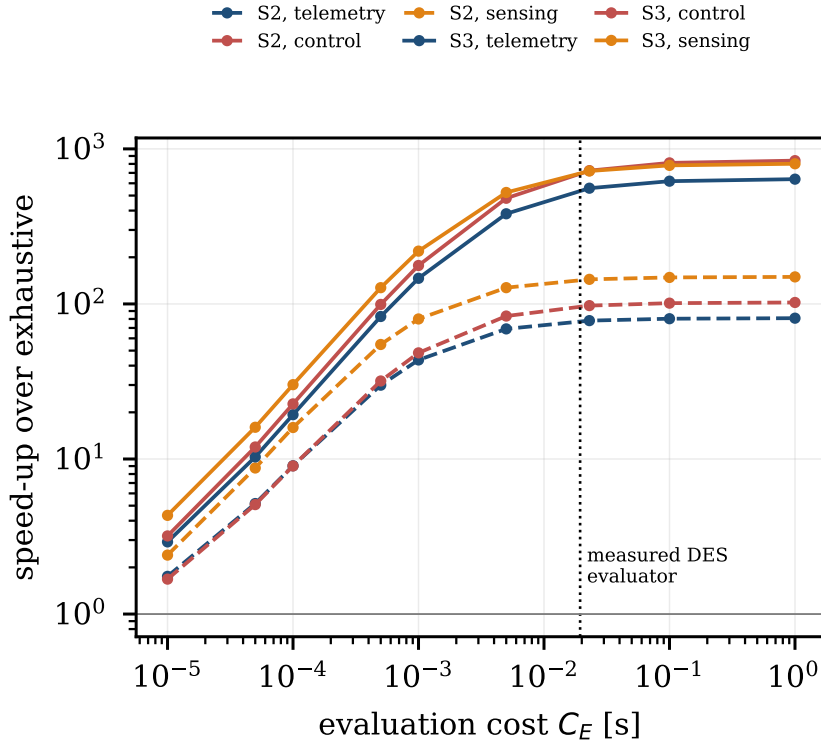


Figure 6: Speed-up over exhaustive enumeration projected as a function of the evaluator cost C_E from the measured overheads and evaluation counts. The dotted line marks the measured cost of the discrete-event evaluator.

6.4. RQ4: comparison with budget-matched exploration

Table 6 reports the comparison over 20 seeds. On S3 NSGA-II reaches a median Pareto recall of $0.50\text{--}0.74$ and an HV ratio of $0.94\text{--}0.98$ only at a budget of 1000 full evaluations, while random

search never recovers a single exact front point at that scale. HCA-DSE attains recall, precision and HV ratio of 1.0 with 41 to 54 evaluations, i.e. between $18\times$ and $24\times$ fewer evaluations than the best budget of the stochastic baseline. At the smaller S2 scale NSGA-II is competitive in hypervolume (0.999 at $B = 1000$) but still misses 9–22% of the exact front points. Fig. 7 shows the seed-level distributions, and Table 7 reports the non-parametric comparison between the two stochastic baselines. At $B = 500$ and $B = 1000$, NSGA-II has a higher median HV ratio for each workload, but none of the one-sided comparisons reaches $p < 0.05$ ($p = 0.054$ – 0.338); Cliff’s δ ranges from 0.08 to 0.30. These samples support a descriptive trend, not a statistically established advantage. At $B = 100$, random search has a higher median HV ratio for all three workloads. The full budget sweep and statistical results are retained in Supplementary Tables S1–S3.

The comparison should be read as a statement about guarantees, not about the quality of evolutionary search: NSGA-II approaches the front as the budget increases in these experiments, but it cannot certify completeness, whereas HCA-DSE returns the exact front and does so at a small fraction of the evaluation budget — provided admissible bounds exist.

Table 6: Budget-matched comparison. Medians over 20 seeds for the stochastic methods at $B = 1000$; the full sweep is in Supplementary Tables S1–S2.

scale	workload	method	evals	recall	HV ratio	IGD ⁺	time [s]
S2	telemetry	exhaustive	3 888	1.000	1.000	0.0000	0.059
		random ($B=1000$)	1000	0.185	0.899	0.0540	0.011
		NSGA-II ($B=1000$)	1000	0.907	0.999	0.0005	0.207
		HCA-DSE	48	1.000	1.000	0.0000	0.044
S2	control	exhaustive	3 888	1.000	1.000	0.0000	0.056
		random ($B=1000$)	1000	0.160	0.897	0.0530	0.012
		NSGA-II ($B=1000$)	1000	0.780	0.999	0.0030	0.234
		HCA-DSE	38	1.000	1.000	0.0000	0.044
S2	sensing	exhaustive	3 888	1.000	1.000	0.0000	0.041
		random ($B=1000$)	1000	0.188	0.868	0.0732	0.009
		NSGA-II ($B=1000$)	1000	0.875	1.000	0.0017	0.270
		HCA-DSE	26	1.000	1.000	0.0000	0.024
S3	telemetry	exhaustive	34 560	1.000	1.000	0.0000	0.372
		random ($B=1000$)	1000	0.000	0.653	0.1500	0.009
		NSGA-II ($B=1000$)	1000	0.500	0.945	0.0466	0.235
		HCA-DSE	54	1.000	1.000	0.0000	0.156
S3	control	exhaustive	34 560	1.000	1.000	0.0000	0.349
		random ($B=1000$)	1000	0.000	0.652	0.1488	0.009
		NSGA-II ($B=1000$)	1000	0.648	0.983	0.0236	0.237
		HCA-DSE	41	1.000	1.000	0.0000	0.170
S3	sensing	exhaustive	34 560	1.000	1.000	0.0000	0.384
		random ($B=1000$)	1000	0.000	0.476	0.2061	0.009
		NSGA-II ($B=1000$)	1000	0.741	0.944	0.1677	0.259
		HCA-DSE	43	1.000	1.000	0.0000	0.107

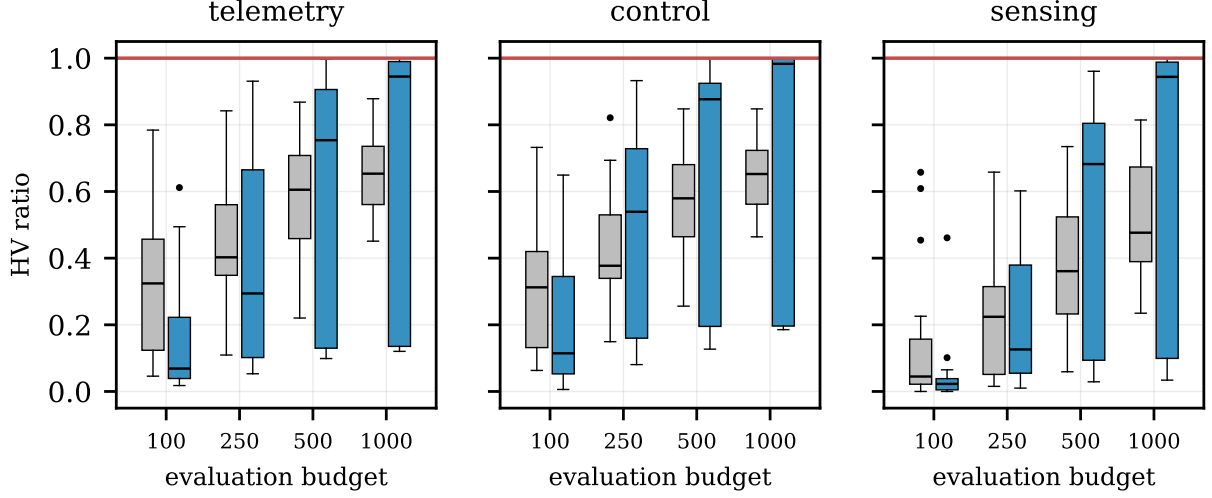


Figure 7: Hypervolume ratio against the exhaustive front on S3 over 20 seeds. Grey: random search; blue: NSGA-II. The red line marks the exact front recovered by HCA-DSE.

Table 7: NSGA-II versus random search on S3 at $B = 500$ and 1000 over 20 seeds: median HV ratio, one-sided Mann–Whitney U p -value and Cliff’s δ . HCA-DSE attains HV ratio 1.0 deterministically with the number of evaluations given in the last column.

workload	budget	NSGA-II HV med.	random HV med.	p	Cliff’s δ	NSGA-II recall	HCA-DSE evals
telemetry	500	0.7539	0.6053	0.338	0.08	0.258	54
telemetry	1000	0.9448	0.6535	0.299	0.1	0.5	54
control	500	0.8767	0.5795	0.082	0.26	0.315	41
control	1000	0.983	0.6524	0.054	0.3	0.648	41
sensing	500	0.6823	0.3609	0.205	0.155	0.185	43
sensing	1000	0.9439	0.4763	0.142	0.2	0.741	43

6.5. Ablation: what each layer contributes

Table 8 and Fig. 8 isolate the layers on S3. Structural pruning alone leaves the full 34 560 well-formed candidates to be evaluated. Feasibility bounds alone reduce this to 7 025–12 700 ($2.7\times$ – $4.9\times$). Dominance bounds alone are far more aggressive, leaving 327–415 evaluations, but the two are complementary: combined they leave 41 to 54, i.e. another $6\times$ – $10\times$ beyond dominance alone, because feasibility pruning removes exactly the sub-trees for which the dominance test would otherwise have to be repeated deep in the tree. All variants preserve the exact front.

Table 8: Ablation on S3. For each workload: visited partial states, full evaluations and runtime.

variant	telemetry			control			sensing		
	states	evals	[ms]	states	evals	[ms]	states	evals	[ms]
structural only	56 797	34 560	424	56 797	34 560	400	56 797	34 560	355
structural + feasibility bounds	22 205	12 700	451	16 110	7 025	330	21 084	10 956	334
structural + dominance bounds	15 557	327	485	15 648	407	412	7 961	415	214
HCA-DSE (all layers)	5 832	54	166	5 826	41	147	4 393	43	118

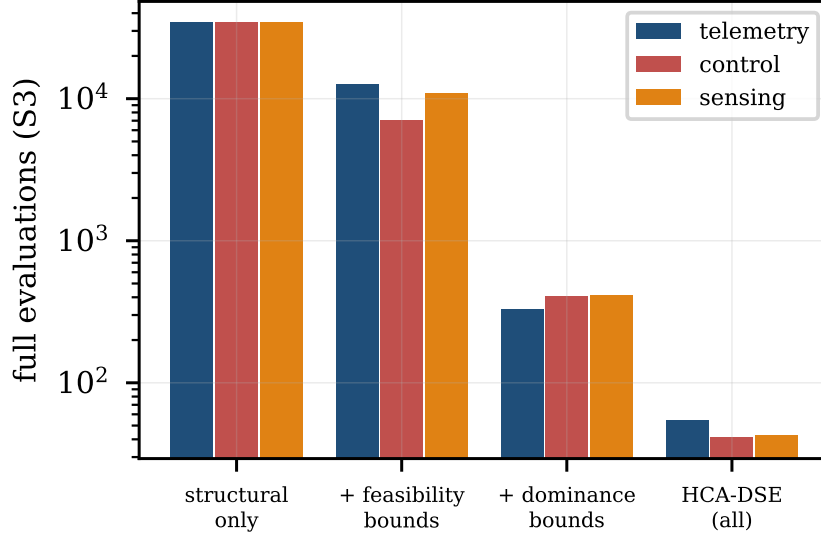


Figure 8: Full evaluations per pruning configuration on S3 (logarithmic scale).

6.6. RQ5: scalability

Table 9 and Fig. 9 extend the raw design space over five orders of magnitude, from 1.3×10^3 to 8.3×10^7 candidates. For the telemetry workload the number of visited partial states grows from 659 to 16 778, a factor of 25, while the raw space grows by a factor of 64 000; over all workloads and scales the visited count never exceeds 16 781. The number of full evaluations is essentially flat, between 18 and 61 across the whole sweep; the runtime stays below 2.4s and the peak Python memory traced during search below 90 KiB, because the Cartesian product is never materialised. The ratio of visited partial states to raw candidates falls from 0.51 to 1.7×10^{-4} .

Table 9: Scalability. For each workload: visited partial states, full evaluations and runtime [s].

$\mathcal{X}$	telemetry			control			sensing		
	states	evals	[s]	states	evals	[s]	states	evals	[s]
1.3×10^3	659	43	0.07	660	33	0.06	391	18	0.03
5.8×10^3	1994	57	0.23	1998	53	0.23	1142	30	0.10
5.2×10^4	7291	60	0.96	7294	56	0.92	5183	53	0.55
3.5×10^5	10346	61	1.39	10349	57	1.34	8643	59	0.95
2.8×10^6	14081	61	1.91	14084	57	1.88	12056	61	1.40
1.6×10^7	15618	61	2.14	15621	57	2.06	13216	61	1.59
8.3×10^7	16778	61	2.31	16781	57	2.23	13888	61	1.72

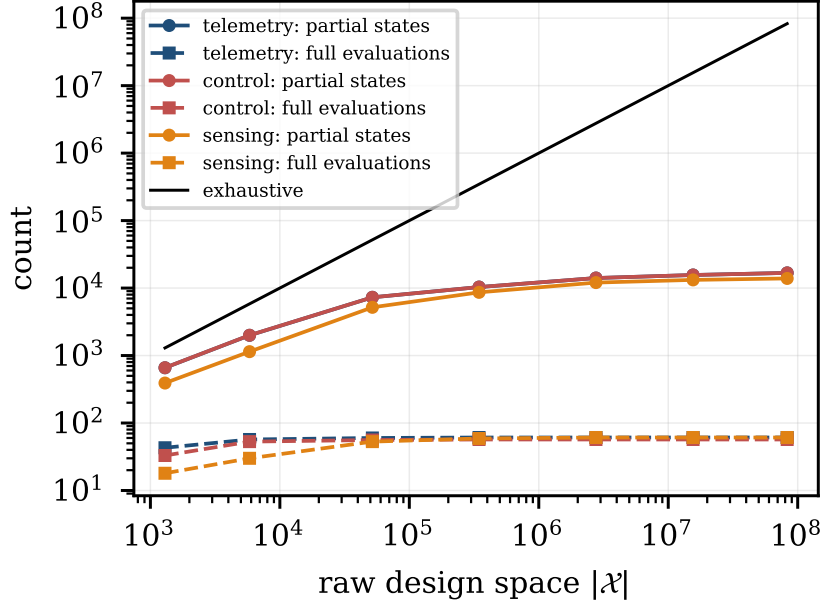


Figure 9: Visited partial states and full evaluations against the raw design-space size.

6.7. RQ6: transfer to a real deployment

Against the discrete-event evaluator the analytical model attains Spearman’s ρ of 0.92, 0.92 and 0.96 for telemetry, control and sensing, with MAPE of 39%, 44% and 14%. On the containerised testbed (Table 10, Fig. 10) the prediction error over the 36 architectures is 28.7% (MAPE) with a median absolute error of 7.5 ms. Within each workload the rank correlation is $\rho = 0.84$ (telemetry), 0.87 (control) and 0.95 (sensing). The pooled correlation is higher ($\rho = 0.97$, $\tau = 0.89$), but part of that value comes from the separation *between* workloads — sensing latencies are an order of magnitude above the other two — so we report it for completeness and base the argument on the per-workload values and on the decision-relevant metric, the pairwise ordering agreement: pooling workloads, among the 498 architecture pairs that the model separates by more than 25%, the measured order agrees in 98.2% of the cases; among the 561 pairs separated by more than 10%, in 96.8%. Within workloads, agreement at the 25% threshold is 89.5% (telemetry; 38 pairs), 90.9% (control; 44 pairs), and 95.7% (sensing; 23 pairs). Thus, the pooled 98.2% also benefits from between-workload separation.

Two systematic effects are visible. For the data-intensive sensing workload the model *over-*predicts latency by roughly 30% — a measured-to-predicted ratio of 0.68–0.73 — on nine of the twelve architectures, the remaining three being exactly the unstable designs discussed next; because the bias is systematic rather than random, the rank correlation for that workload is the highest of the three ($\rho = 0.95$) while its MAPE is the worst (50%). And four architectures — all sensing designs with $\rho \geq 0.75$ — were not reproducible across repeats, varying by factors of two to five (for instance 157, 293 and 556 ms against a predicted 82 ms). Near saturation neither the analytical model nor the deployment itself is stable, which is a result about the limits of design-time prediction rather than measurement noise to be filtered out.

Table 10: Testbed validation. Pair agreement is the share of architecture pairs whose measured order matches the model order, among pairs the model separates by more than 25%. ALL pools workloads. Full subsets and 10% results appear in Supplementary Table S4.

workload	n	MAPE [%]	Spearman ρ	agreement > 25%	pairs
telemetry	12	20.3	0.839	0.895	38
control	12	15.7	0.872	0.909	44
sensing	12	50.0	0.949	0.957	23
ALL	36	28.7	0.97	0.982	498

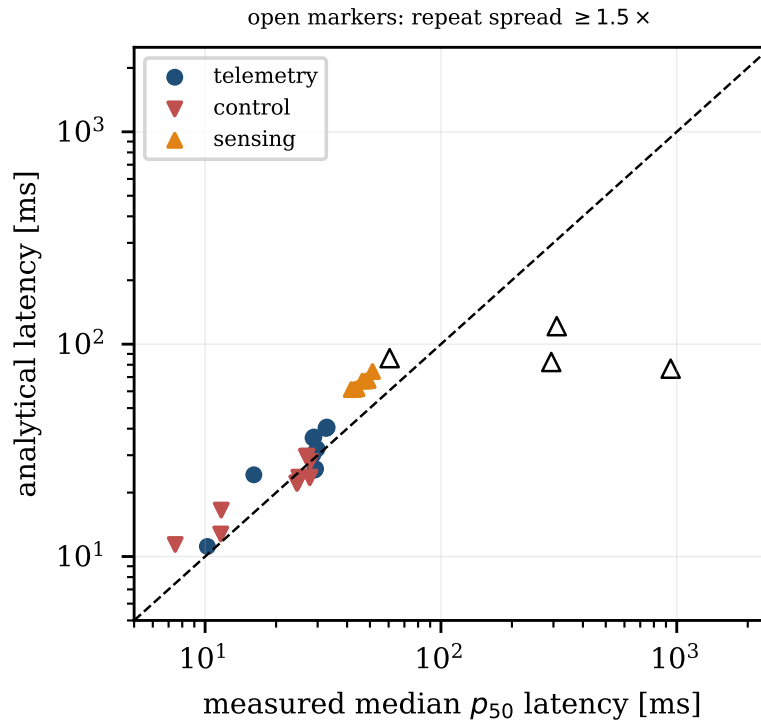


Figure 10: Analytical latency against the measured median p_{50} latency of the same architecture on the containerised testbed. Open markers denote the four architectures whose repeats differ by a factor of 1.5 or more.

6.8. A model correction found by the measurements

The following is an exploratory, post-hoc correction and is reported as such: the comparison below uses the same testbed dataset that revealed the problem, so the improvement is a consistency check rather than an out-of-sample validation. The first version of the analytical model used an M/M/1 waiting term. Because stage execution is deterministic, that formulation overestimates waiting by a factor of two, and replacing it with the M/D/1 expression of (3) reduced the MAPE from 28.9% to 23.7% on an identical set of measurements. Interestingly the rank correlation was slightly *higher* for the M/M/1 variant (0.918 versus 0.889 on that testbed configuration), i.e. accuracy and ordering do not improve in lockstep — a practical reminder that a model tuned for absolute prediction is not automatically the better model for exploration.

7. Discussion

7.1. *Why hierarchical pruning works here*

The results of Section 6 are driven by a property of the design space rather than by a clever search order: in a heterogeneous edge–cloud CPS the vast majority of candidate architectures is either ill-formed, or violates a hard constraint that is already implied by a handful of early decisions, or is dominated by a design that is cheaper in every objective. On S3 only 34 560 of 51 840 candidates are even well-formed, of those a further 7 000 to 12 700 survive the feasibility bounds, and of those fewer than 60 need to be evaluated. Every candidate removed at those layers would otherwise consume a full simulation.

The ablation makes the mechanism explicit. Dominance pruning is the stronger of the two bound-based layers, but it is only effective once an archive of good designs exists; feasibility pruning removes precisely the sub-trees in which no such design will ever be found, so it reduces the number of tree nodes at which the dominance test has to be evaluated. The combination is roughly an order of magnitude stronger than either layer alone.

7.2. *When HCA-DSE is the right tool — and when it is not*

The method is advantageous when the design space is large, the constraints are tight, the evaluator is expensive, and admissible bounds can be constructed. The crossover analysis makes the third condition quantitative: with the analytical evaluator, whose cost is comparable to that of the bounds themselves, the wall-clock gain is only 1.3–3.3 $\times$ despite a 99.9% reduction in evaluations, whereas with a discrete-event evaluator the cost model projects gains of up to two orders of magnitude. Since industrial dCPS exploration reports evaluation costs of up to a minute per design point [1], the practically relevant regime is firmly on the favourable side of the crossover.

Conversely, the method offers little when the space is small, the constraints are loose, the evaluation is nearly free, or — most importantly — when no admissible bound can be constructed for the objectives of interest. If a bound cannot be proven admissible it must not be used: the guarantee of Theorem 1 rests entirely on that property, and an inadmissible bound silently converts an exact method into an approximate one without any indication in the output.

7.3. *Relation to symbolic exact exploration*

Symbolic approaches [4] prune partial designs with a constraint solver and are, in principle, more general than closed-form bounds: any constraint the solver can express can participate in the pruning. The assignment-monotone construction of the cited method does not directly supply our queueing bounds. Our construction instead uses objective-specific relaxations. These bounds preserve the analytical front, although the evaluated variable order activates the exact queueing term only at full assignment (Remark 1). A solver-based front end and an earlier assignment of queueing determinants are possible extensions whose performance remains untested.

7.4. *Relation to evaluator-centric methodologies*

CompDSE [1] and HCA-DSE answer different questions. The former asks how a design point of an industrial dCPS can be modelled and evaluated faithfully; the latter asks which design points need not be evaluated at all. Their combination is conditional: analytical bounds would need to be proved admissible for the trace-derived simulator itself. Admissibility for the analytical model and good empirical rank agreement do not establish that condition. The crossover analysis is a cost projection and supplies no simulator-level exactness guarantee.

7.5. Accuracy versus ordering

The empirical validation supports a narrower claim than "the model is accurate". The model's absolute error is 16–50% depending on the workload, which is in the same range as the 20% execution-time deviation reported for a design-time placement tool validated on a real deployment [15]. What the exploration requires is weaker: that the model order architectures correctly, and that claim is supported much more strongly — 89.5–95.7% within-workload agreement on pairs separated by more than 25% (98.2% when pooled). The distinction must be stated precisely, because it bounds what Theorem 1 buys. The theorem guarantees that HCA-DSE returns the exact Pareto front of the analytical model; correct ordering makes that model a useful design-space surrogate, but it does not establish that its front coincides with the Pareto front of the deployed system. A design-time method can be exact with respect to its model and still inherit every error of that model, and no amount of pruning correctness repairs a misspecified objective. The complement is equally informative — within a single workload the rank correlation is lower, because after the M/D/1 correction many designs of one workload differ by less than the measurement noise. A model that cannot separate near-equivalent architectures is not a defect of the exploration method but an argument for the two-level scheme: analytical exploration can select candidates for high-fidelity reassessment, but discarding all other candidates preserves the high-fidelity front only if bounds are also proved admissible for that evaluator.

8. Threats to validity

Internal validity. The correctness of the method rests on the admissibility of the bounds. We address this at three levels: proofs (Lemmas 1–2, Propositions 1–3, Theorem 1); an exhaustive machine check of $LB_i(p) \leq f_i(x)$ over all prefixes and all completions on S1 for the three workloads; and a direct check that, for every pruned prefix, no completion contradicts the claim of the corresponding rule. We deliberately separate *theoretical* correctness from *implementation* correctness and verify both, since an admissible bound that is implemented incorrectly is indistinguishable, in the output, from an inadmissible one. Structural pruning is part of the definition of the design space rather than a consequence of the constraints, and the exhaustive baseline uses the same predicate; this was discovered by the pruning-safety test and is stated explicitly rather than absorbed into the comparison.

Post-hoc model correction. The replacement of the M/M/1 waiting term by the M/D/1 expression was motivated by the testbed measurements and evaluated on the same dataset. Deterministic stage execution motivates the factor-of-two change under an idealised single-server queue, but does not establish an exact queueing model for the full deployment; the reported improvement in MAPE is an in-sample consistency check, not an out-of-sample validation, and is labelled as such in Section 6.

Construct validity. The analytical model uses per-node service times with a stage-wise M/D/1-inspired waiting multiplier based on average tier utilisation. It is neither an exact aggregate-server queue nor an exact multi-server queue, and its error has no proved sign. Transport protocol effects are ignored, the power model is linear in utilisation, and costs are normalised units. Testbed validation quantifies latency error and ordering agreement, but does not prove admissibility for a simulator or for the deployment. The sensing workload exposes the main structural gap — for large payloads over a shaped link the model does not capture transport-level effects and over-predicts systematically. The energy and cost models are not validated against measurements at all; they are used for comparative exploration only, and no absolute energy or cost claim is made.

External validity. The workloads are synthetic, although chosen to be qualitatively different rather than rescaled versions of one another, and the architecture template is fixed to four tiers with three stages. The empirical study is further restricted to configurations that fit on a single machine (≤ 4 edge nodes, ≤ 2 gateways), and link shaping is applied in the forward direction of each hop, matching the one-way transfers counted by the model. Results on a geographically distributed deployment may differ, particularly for the WAN hop.

Conclusion validity. Deterministic methods (exhaustive, HCA-DSE) require no statistical treatment. Stochastic baselines are run with 20 independent seeds and compared with non-parametric tests (Mann–Whitney U , Cliff’s δ). Testbed configurations are repeated three times and reported by their median; the four configurations whose repeats differ by more than a factor of 1.5 are reported separately rather than discarded, and all four operate at $\rho \geq 0.75$.

Evaluation-budget accounting. The archived stochastic comparisons use the counters reported by the exploration implementations. In random search a draw consumes budget even if structural validation rejects it before the evaluator is called. Reported budgets therefore need not equal expensive evaluator invocations, and evaluation-saving ratios against stochastic baselines must be interpreted with this accounting limitation. A comparison with separately instrumented proposal, rejection and evaluator-call counts is needed to establish strict matching of expensive evaluation work.

Limitations. The method assumes a static workload during one exploration, does not model runtime migration or autoscaling, and requires bounds to be derived per objective — a manual step that must be repeated when the objective set changes. No comparison against an exact symbolic or mixed-integer formulation is provided. Exact integer programming is demonstrably practical for allocation problems in this domain [13, 14], but no equivalent solver encoding of our finite-domain nonlinear model has been implemented. Approximate linearisation would change the objective; exact table, disjunctive or suitable nonlinear encodings are alternatives whose computational cost has not been measured. This missing baseline limits comparative claims of algorithmic novelty and efficiency.

9. Conclusions

Architecture exploration for heterogeneous distributed cyber-physical systems is limited less by the size of the design space than by the cost of evaluating a single design point. This paper presented HCA-DSE, a hierarchical constraint-aware exploration method that removes structurally ill-formed, provably infeasible and provably dominated regions of the space before any full evaluation is performed. Its technical core is a family of admissible bounds for *contention-dependent* objectives, obtained by a term-wise relaxation with a queueing term that becomes exact once all its determinants are assigned. In the evaluated order this occurs at full assignment. The construction does not require assignment-monotone objective costs, and we proved that it preserves the exact analytical Pareto front.

Across four design-space scales, three workload classes and spaces of up to 8.3×10^7 candidate architectures, HCA-DSE performed 16 to 61 full evaluations, with full-evaluation reductions of 99.90% to 99.98% on S3–S4, while reproducing the exhaustive Pareto front exactly wherever exhaustive enumeration is tractable. Both the admissibility of the bounds and the equality of the fronts were verified exhaustively by machine. Budget-matched NSGA-II required 1000 evaluations to reach a

median Pareto recall of 0.50–0.91. The break-even evaluator cost is in the microsecond range, and using the measured per-design cost of the discrete-event evaluator, the cost model projects a speed-up of up to $706\times$. On a containerised edge–gateway–cloud testbed with per-node link shaping, 36 architectures and 108 measured runs, the analytical ordering agreed with the measured ordering in 98% of the pooled architecture pairs that the model separates by more than 25%.

The method is intended as design-time decision support: it does not require runtime adaptation, learned performance models, or a high-fidelity simulator in the inner loop. Its natural continuation is the combination with an evaluator-centric methodology, in which analytical bounds reduce an industrial design space to a few dozen candidates and a trace-derived simulator evaluates only those; and the derivation of tighter, possibly surrogate-assisted bounds that remain provably admissible.

Data availability

The complete implementation, all experiment scripts, the testbed and every result file required to regenerate the figures and tables of this paper are available in the replication package accompanying the submission.

References

- [1] F. S. Saadatmand, T. Stefanov, I. González Alonso, A. D. Pimentel, B. Akesson, CompDSE: A methodology for design space exploration of computing subsystems within complex cyber-physical systems, *IET Cyber-Physical Systems: Theory & Applications* 10 (1) (2025) e70019. doi:10.1049/cps2.70019.
- [2] M. Herget, F. S. Saadatmand, M. Bor, I. González Alonso, T. Stefanov, B. Akesson, A. D. Pimentel, Design space exploration for distributed cyber-physical systems: State-of-the-art, challenges, and directions, in: 25th Euromicro Conference on Digital System Design (DSD), 2022, pp. 632–640. doi:10.1109/DSD57027.2022.00090.
- [3] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197. doi:10.1109/4235.996017.
- [4] K. Neubauer, P. Wanko, T. Schaub, C. Haubelt, Exact multi-objective design space exploration using ASPmT, in: Design, Automation & Test in Europe Conference (DATE), 2018, pp. 257–260. doi:10.23919/DATE.2018.8342014.
- [5] C. Haubelt, K. Neubauer, T. Schaub, P. Wanko, Design space exploration with answer set programming, *KI - Künstliche Intelligenz* 32 (2018) 205–206. doi:10.1007/s13218-018-0530-3.
- [6] S. Petchrompo, D. W. Coit, A. Brintrup, A. Wannakrairot, A. K. Parlikad, A review of Pareto pruning methods for multi-objective optimization, *Computers & Industrial Engineering* 167 (2022) 108022. doi:10.1016/j.cie.2022.108022.
- [7] L. Mandow, J. L. Pérez-de-la Cruz, Multiobjective A* search with consistent heuristics, *Journal of the ACM* 57 (5) (2010) 1–25. doi:10.1145/1754399.1754400.
- [8] K. Neubauer, B. Beichler, C. Haubelt, Exact design space exploration based on consistent approximations, *Electronics* 9 (7) (2020) 1057. doi:10.3390/electronics9071057.

- [9] B. S. Stewart, C. C. White, Multiobjective A*, *Journal of the ACM* 38 (4) (1991) 775–814. doi:10.1145/115234.115368.
- [10] B. Kienhuis, E. F. Deprettere, P. van der Wolf, K. Vissers, A methodology to design programmable embedded systems: The Y-chart approach, in: *Embedded Processor Design Challenges*, Springer, 2002, pp. 18–37. doi:10.1007/3-540-45874-3_2.
- [11] A. Varga, R. Hornig, An overview of the OMNeT++ simulation environment, in: *1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008, pp. 1–10. doi:10.4108/ICST.SIMUTOOLS2008.3027.
- [12] Y. Xiao, C. Oh, M. Lora, P. Nuzzo, Efficient exploration of cyber-physical system architectures using contracts and subgraph isomorphism, in: *Design, Automation & Test in Europe Conference (DATE)*, 2024, pp. 1–6. doi:10.23919/DATE58400.2024.10546764.
- [13] A. Kouloumpris, G. L. Stavrinides, M. K. Michael, T. Theocharides, An optimization framework for task allocation in the edge/hub/cloud paradigm, *Future Generation Computer Systems* 155 (2024) 354–366. doi:10.1016/j.future.2024.02.005.
- [14] A. Kouloumpris, G. L. Stavrinides, M. K. Michael, T. Theocharides, A reliability- and latency-driven task allocation framework for workflow applications in the edge–hub–cloud continuum, *Future Generation Computer Systems* 180 (2026) 108414. doi:10.1016/j.future.2026.108414.
- [15] H. Sedghani, F. Filippini, D. Ardagna, SPACE4AI-D: A design-time tool for AI applications resource selection in computing continua, *IEEE Transactions on Services Computing* 17 (6) (2024) 4324–4339. doi:10.1109/TSC.2024.3479935.
- [16] J. Blank, K. Deb, pymoo: Multi-objective optimization in Python, *IEEE Access* 8 (2020) 89497–89509. doi:10.1109/ACCESS.2020.2990567.
- [17] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, *IEEE Transactions on Evolutionary Computation* 3 (4) (1999) 257–271. doi:10.1109/4235.797969.
- [18] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, Modified distance calculation in generational distance and inverted generational distance, in: *Evolutionary Multi-Criterion Optimization*, Springer, 2015, pp. 110–125. doi:10.1007/978-3-319-15892-1_8.
- [19] C. Audet, J. Bignon, D. Cartier, S. Le Digabel, L. Salomon, Performance indicators in multiobjective optimization, *European Journal of Operational Research* 292 (2) (2021) 397–422. doi:10.1016/j.ejor.2020.11.016.
- [20] T. Blickle, J. Teich, L. Thiele, System-level synthesis using evolutionary algorithms, *Design Automation for Embedded Systems* 3 (1) (1998) 23–58. doi:10.1023/A:1008899229802.
- [21] C. Haubelt, J. Teich, Accelerating design space exploration using Pareto-front arithmetics, in: *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2003, pp. 525–531. doi:10.1145/1119772.1119882.

- [22] M. Lukasiwycz, M. Glaß, C. Haubelt, J. Teich, Efficient symbolic multi-objective design space exploration, in: Asia and South Pacific Design Automation Conference (ASP-DAC), 2008, pp. 691–696. doi:10.1109/ASPdac.2008.4484040.
- [23] L. Kleinrock, Queueing Systems, Volume 1: Theory, Wiley, New York, 1975.
- [24] M. Harchol-Balter, Performance Modeling and Design of Computer Systems: Queueing Theory in Action, Cambridge University Press, 2013. doi:10.1017/CB09781139226424.
- [25] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.
- [26] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychological Bulletin 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
- [27] S. Hemminger, Network emulation with NetEm, in: Linux Conference Australia (linux.conf.au), 2005.